



FlexLearn: Fast and Highly Efficient Brain Simulations Using Flexible On-Chip Learning

Eunjin Baek^{1,*}, Hunjun Lee^{1,*}, Youngsok Kim², and Jangwoo Kim^{1,‡}

¹Department of Electrical and Computer Engineering, Seoul National University

²Department of Computer Science, Yonsei University

{ebaek, hunjunlee7515}@snu.ac.kr, youngsok@yonsei.ac.kr, jangwoo@snu.ac.kr

ABSTRACT

To understand how the human brain works, neuroscientists heavily rely on brain simulations which incorporate the concept of time to their operating model. In the simulations, neurons transmit their signals through synapses whose weights change over time and by the activity of the associated neurons. Such changes in synaptic weights, known as learning, are thought to contribute to memory, and various learning rules exist to model different behaviors of the human brain. Due to the diverse neurons and learning rules, neuroscientists perform the simulations using highly programmable general-purpose processors. Unfortunately, the processors greatly suffer from the high computational overheads of the learning rules. As an alternative, brain simulation accelerators achieve orders of magnitude higher performance; however, they have limited flexibility and cannot support the diverse neurons and learning rules.

In this paper, we present *FlexLearn*, a flexible on-chip learning engine to enable fast and highly efficient brain simulations. *FlexLearn* achieves high flexibility by supporting diverse biologically plausible sub-rules which can be combined to simulate various target learning rules. To design *FlexLearn*, we first identify 17 representative sub-rules which adjust the synaptic weights in different manners. Then, we design and compact the specialized datapaths for the sub-rules and identify dependencies between them to maximize parallelism. After that, we present an example flexible brain simulation processor by integrating the datapaths with the state-of-the-art flexible digital neuron and existing accelerator to support end-to-end simulations. Our evaluation using a 45-nm cell library shows that the 128-core brain simulation processor prototype with *FlexLearn* greatly improves the harmonic mean per-area performance and the energy efficiency by 30.07× and 126.87×, respectively, over the server-class CPU. The prototype also achieves the harmonic mean per-area speedup of 1.41× over the current state-of-the-art 128-core accelerator which supports programmable learning rules.

*Both authors contributed equally to this work.

‡Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MICRO-52, October 12–16, 2019, Columbus, OH, USA

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6938-1/19/10...\$15.00

<https://doi.org/10.1145/3352460.3358268>

CCS CONCEPTS

• **Hardware** → **Neural systems; Integrated circuits.**

KEYWORDS

brain simulations, neuromorphic accelerators, on-chip learning

ACM Reference Format:

Eunjin Baek, Hunjun Lee, Youngsok Kim, and Jangwoo Kim. 2019. FlexLearn: Fast and Highly Efficient Brain Simulations Using Flexible On-Chip Learning. In *The 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-52)*, October 12–16, 2019, Columbus, OH, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3352460.3358268>

1 INTRODUCTION

Neuroscientists strive to understand the working mechanism of a human brain, the central organ of the human nervous system consisting of about 86 billion neurons [8]. To study the complex human brain, neuroscientists heavily rely on simulations which incorporate the concept of time to their operating models to closely mimic the temporal behaviors of the human brain. In the simulations, a neuron transmits a signal to other neurons via synapses when its internal state satisfies certain conditions. The signal affects not only the destination neurons' internal states but also the weights of the associated synapses. Such temporal changes in the weights are called *learning* (or synaptic plasticity) and are widely considered as an underlying mechanism of memory [60]. Taking the temporal aspects into account allows neuroscientists to understand and mimic the mechanisms of the human brain (e.g., object and speech recognition [15, 90]).

The human brain consists of a variety of highly complex neurons and learning rules. As a result, brain simulations demand fast and highly flexible hardware capable of simulating the diverse neurons and learning rules. First, the synaptic weights of different synapses change differently [64]. For example, upon receiving a signal from the source neurons, some synapses temporarily change their weights for only a short period of time (lasting up to few seconds), whereas some other synapses memorize the signal by making the weight changes permanent. Second, the behaviors of the neurons may behave differently even for the same set of input signals during the same amount of time [45]. The internal state of a neuron may be stable over time, whereas another neuron's state may decay over time and eventually reach the lowest possible level if no input signals are received. As a result, demystifying the human brain demands fast and highly flexible hardware capable of simulating the diverse neurons and learning rules.

To satisfy the flexibility and high performance demands, neuroscientists typically rely on general-purpose processors offering high

programmability (e.g., CPUs [11, 29, 32, 38], GPUs [21, 41, 66, 101]) or high-performance accelerators [9, 18, 71, 79]. Unfortunately, their limited performance and flexibility make difficult for neuroscientists to achieve fast and highly efficient brain simulations. On one hand, CPU-based accelerators (e.g., SpiNNaker [71]) provide high flexibility to simulate any neurons and learning rules; however, they suffer from the low performance CPU cores and cannot achieve high-performance simulations. On the other hand, the accelerators like Intel’s Loihi [18] achieve high performance by optimizing their designs for brain simulations. Unfortunately, the accelerators are inflexible so that they can simulate only a limited set of neurons and learning rules. A recent study [53] proposes a flexible digital neuron capable of simulating the diverse neuron models; however, it only supports limited part of the brain simulation (i.e., neuron computation). Therefore, neuroscientists desperately demand a highly flexible accelerator supporting the diverse neurons and learning rules.

In this paper, we present FlexLearn, a highly flexible on-chip learning architecture to support biologically plausible learning rules. FlexLearn can be seamlessly augmented to existing neuron models for enabling fast and highly efficient brain simulations. To design FlexLearn, we first identify 17 representative sub-rules from an extensive analysis of prior neuroscience publications on learning. The identified sub-rules serve as the primitives of the diverse learning rules; a learning rule is essentially a combination of the sub-rules. After classifying the sub-rules into six categories based on their biological characteristics, we design per-category datapaths which serve as the basic building blocks of FlexLearn. Next, to reduce the hardware costs, we compact the datapaths by exploiting the common sub-expressions shared among the sub-rules. To improve the performance of the datapaths, we identify dependencies between the categories and maximize parallelism between them. The resulting design, FlexLearn, is capable of simulating the diverse learning rules through microcodes by selectively activating the necessary datapaths.

To build an example flexible brain simulation processor using FlexLearn, we implement a 128-core brain simulation processor by integrating FlexLearn with Flexon [53], the state-of-the-art flexible neuron architecture. We synthesize its RTL code using a 45-nm cell library for area, power, and latency evaluations. To compare our processor against a convincing baseline, we carefully implement a baseline 128-core brain simulation processor based on an end-to-end industry solution which employs neuron and learning engine, but with limited flexibility [18]. Our synthesis results show that our processor design incurs only 6.25% additional area over the baseline. To evaluate the performance and energy efficiency, we use three biological neural networks used in real brain simulations with four different learning rule combinations as our benchmarks. The results show that our processor improves the harmonic mean per-area speed and the energy efficiency by $30.07\times$ (40.32 for speed) and $126.87\times$, respectively, over the CPU. Compared to the brain simulation processor baseline, our FlexLearn-based processor improves the per-area speedup and the energy efficiency by $1.41\times$ ($1.50\times$ for speedup) and $1.17\times$, respectively.

In summary, our work makes the following contributions:

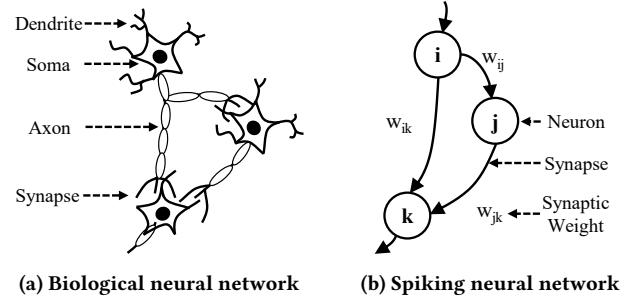


Figure 1: Biological neurons and synapses, and the abstraction of the neurons and the synapses in SNNs

- **Identification of the Target Learning (Sub-)Rules:** From an extensive analysis of neuroscience publications, we identify 17 representative sub-rules which can simulate diverse learning rules. Our analysis clarifies what FlexLearn should implement for high flexibility, and can guide future research on brain simulation.
- **FlexLearn Architecture:** We present FlexLearn, a highly flexible and efficient on-chip learning architecture for brain simulations. FlexLearn is not only highly flexible by supporting the identified learning rules, but also highly efficient by exploiting the common sub-expressions and dependencies among the sub-rules.
- **Example Brain Simulation Processor:** We design an example brain simulation processor by integrating FlexLearn with Flexon, the state-of-the-art flexible digital neuron to realize both flexible, fast and highly efficient end-to-end simulations.

2 BACKGROUND

2.1 Organization of the Human Brain

In the human brain, neurons transmit electrochemical signals, called spikes, over synapses, chemical channels between neurons, to communicate with each other. A neuron consists of three major components: dendrites, a soma, and axons (Figure 1a). Dendrites act as receivers of spikes sent from neighboring neurons through synapses. The spikes are then relayed to the soma, the body of the neuron. The soma maintains the neuron’s membrane potential which tracks the temporal history of the received spikes. When the neuron’s membrane potential reaches threshold voltage, a pre-defined threshold, the soma fires a spike to other neurons through the axons. During the process, the membrane potential slowly decays over time and maintains a steady state if no input spikes are received for a long period of time.

When a neuron receives a spike through a synapse, the neuron’s membrane potential changes with respect to the characteristics of the synapse. Synapses are typically classified into two types: excitatory and inhibitory synapses. Excitatory synapses increase the membrane potential whereas inhibitory synapses decrease the membrane potential. In addition to the type of a synapse, the amount of change in the membrane potential depends on the strength of the synapse, known as synaptic weight. As a result, a neuron’s

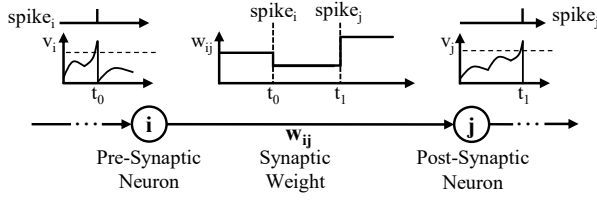


Figure 2: Example working model of synaptic plasticity. Spikes from the pre-synaptic (i) and post-synaptic (j) neurons affect the synaptic weight w_{ij} .

membrane potential tends to depend more heavily on spikes from the neurons connected via synapses with higher weights rather than those from synapses with lower weights. Such spike-triggered changes in the membrane potential incur the complex behaviors of the human brain [34].

Learning, also known as synaptic plasticity, is the ability of synapses to strengthen or weaken over time and is widely believed as an underlying mechanism of memory [43, 58, 60, 74]. The changes in synaptic weights depend on various biological phenomena: spiking activity and membrane voltage of the connected neurons, synaptic weight value, and even the external stimulus. Depending on how each learning rule correlates which phenomena, it mimics different behavior of the brain. Also, various sub-rules explaining different biological phenomena interact with each other to form a single learning rule and explain high level functions of the brain (e.g., memory formation [40, 102]).

2.2 Modeling the Human Brain

As the temporal behaviors of the human brain depend on timing, Spiking Neural Networks (SNNs) which incorporate the concept of time to their operating model are highly useful for brain simulations. An SNN is essentially a directed graph whose nodes are neurons and edges are synapses (Figure 1b). SNNs model a synapse as an edge whose source and destination nodes are pre- and post-synaptic neurons, respectively, and weight is the synaptic weight of the synapse. When the pre-synaptic neuron fires a spike, the spike travels to the post-synaptic neurons through synapses after a pre-defined delay. Upon receiving a spike, the post-synaptic neuron updates its internal state according to its neuron model and the synaptic weight of the synapse the spike passed through.

The neuronal dynamics of biological neurons are characterized by spiking neuron models in SNNs. A neuron maintains a temporal internal state, and sends and receives spikes to and from other neurons, respectively. Spiking neuron models differ by how they model the membrane potential of a neuron. For instance, the Hodgkin-Huxley model [42] expresses a neuron’s membrane potential as a Resistor-Capacitor circuit (RC circuit), whereas Leaky Integrate-and-Fire (LIF) model [86] simplifies the membrane potential as a single state variable. Despite the differences, spiking neuron models respect the basic mechanism of firing a spike when the membrane potential reaches the threshold voltage.

SNNs model synaptic plasticity by updating the weight of a synapse when the pre- and post-synaptic neurons fire spikes. Figure 2 illustrates an example synaptic plasticity mechanism which

updates the synaptic weight of a synapse when the pre- and post-synaptic neurons fire spikes. In the example, the pre-synaptic neuron i fires a spike as its membrane potential reaches its threshold voltage at t_0 . Then, the fired spike goes to the postsynaptic neurons via synapses. When the spike passes the synapse, the synaptic weight w_{ij} decreases (or depresses) depending on the neurons’ internal states. w_{ij} also depends on the activity of the post-synaptic neuron j ; the spike from j fired at t_1 increments (or facilitates) w_{ij} .

2.3 Simulating the Human Brain

As the brain incorporates the concept of time, simulating the brain involves evaluation of the temporal neuronal and synaptic state updates during a fixed amount of time. To enable time-driven simulations, brain simulators utilize a pre-defined timestep Δt which represents the unit of time (e.g., 1 ms in biological time) of a simulation. After initializing the neuronal and synaptic states ($t = 0$), the simulators advance the biological time by one timestep ($t = \Delta t$) by evaluating the neuronal and synaptic dynamics which occurs during one timestep. Spikes generated by the neurons during $[0, \Delta t)$ get propagated to their destination neurons after the delay of their corresponding synapses. In this way, the simulation continues by advancing the biological time by Δt ($t = \Delta t, \Delta t \times 2, \Delta t \times 3$). The simulation terminates when the biological time reaches a pre-defined deadline (e.g., 1,000 timesteps).

For simplicity, we divide a timestep into three stages: neuron computation, synaptic plasticity, and spike propagation.

Neuron Computation: This stage simulates the neuronal dynamics of neurons. As the first step, neurons receive the incoming spikes from the previous timestep and accumulate the spikes according to their neuron models. This stage also triggers a neuron to fire a spike if its membrane potential reaches its threshold voltage. When the neuron fires a spike, the information on the spike is propagated to the next stage for synapses to update their weights.

Synaptic Plasticity: In this stage, the synaptic weights of synapses are updated according to synapses’ learning rules and the information on whether neurons have fired a spike in the current time range. The updated synaptic weights are then forwarded to the next stage called spike propagation stage.

Spike Propagation: This stage is responsible for associating synaptic weights to the spikes and forwarding the spikes to their destination neurons. To do so, this stage first collects the spikes generated by neurons, fetches the synaptic weights of the associated synapses, and accumulates the synaptic weights of the spikes having the same destination neuron. The accumulated weights are then forwarded to the destination neurons after a pre-defined delay.

3 MOTIVATION & DESIGN GOALS

3.1 High Computational Overheads

To examine which of the brain simulation stages contributes the most to the simulation latency, we profile a representative biological neural network used by Brunel [13], with diverse learning rules. The Brunel network consists of 5,750 neurons. The neurons are connected randomly with a 10% probability, resulting in the network having 3,306,250 synapses. Learning rules affect the 2,116,000 synapses; the synaptic weights of the remaining synapses remain constant. We configure the network to use a timestep of 0.1 ms to

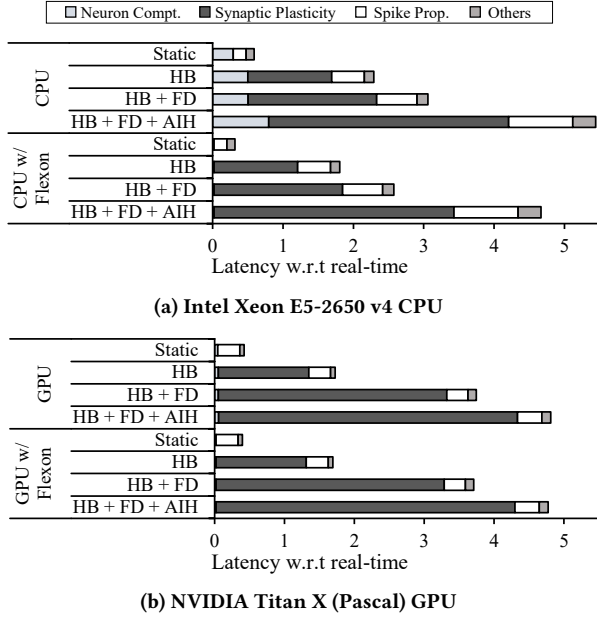


Figure 3: Breakdown of the simulation latency of Brunel network on the server-class CPU and GPU with representative learning rule configurations

run for 1 second of biological time (i.e., 10,000 timesteps). The underlying server-class CPU and GPU are 12-core Intel Xeon E5-2650 v4 CPU [1] and NVIDIA Titan X (Pascal) GPU [2], respectively. We use NEST [29] simulator for the CPU and GeNN [101] for the GPU.

Figure 3 shows the simulation latency breakdown with different learning configurations; Others indicates the unclassified functions (e.g., network construction and random input generation). The profiling results show that the synaptic plasticity stage incurs the highest latency. The details of the learning configurations are discussed in Section 4.1. When the learning configuration enables only one sub-rule (HB), the synaptic plasticity stage contributes to 51.9% and 75.0% of the total latency on the CPU and the GPU, respectively. Enabling more sub-rules increases the contribution of the synaptic plasticity stage to the total latency; the synaptic plasticity stage incurs 88.9% of the total latency on the GPU when three sub-rules are enabled (HB+FD+AIH).

We further profile the simulation latency in case the CPU and the GPU cores offload the neuron computation stage to Flexon [53], the state-of-the-art digital neuron supporting diverse neuron models. When the digital neuron is available, the results show that the synaptic plasticity stage incurs up to 73.0% and 89.6% of the total latency on the CPU and the GPU, respectively. In addition, the simulations lead to 5.4 $\times$ and 4.7 $\times$ higher latency with respect to the real-time on the CPU and the GPU, respectively. (i.e., it takes 5.4 and 4.7 seconds to simulate 1 second of biological time). Such slow simulation speeds are detrimental as various learning rules operate on large timescales (e.g., hours to days [46, 95, 102]). In summary, the synaptic plasticity stage is the major performance bottleneck and should be accelerated to achieve fast and highly efficient simulations.

Table 1: Summary of representative brain simulation chips

Name	Performance	Flexibility		Brain Simulations
		Learnings	Neurons	
SpiNNaker [71]	Low	High	High	✓
TrueNorth [5]	High	Low	Low	✗
Loihi [18]	High	Moderate	Low	✗
Flexon [53]	High	None	High	✗
FlexLearn (Ours)	High	High	High	✓

3.2 Limitations of the Existing Accelerators

As different learning rules work together to explain various phenomena of the human brain, brain simulation accelerators should be highly flexible by supporting the diverse learning rules. Unfortunately, we find that the existing accelerators are either slow or inflexible. Table 1 summarizes the limitations of the representative accelerators. First, to achieve high flexibility, SpiNNaker [71] is equipped with low-power CPU cores; however, its performance is limited by the CPU cores and incurs higher latency than the other accelerators. Second, IBM’s TrueNorth [5] is a configurable neuromorphic processor supporting highly efficient neuron computation. TrueNorth achieves highly efficient neuron computation; however, it lacks on-chip learning support. There are efforts to implement on-chip learning on TrueNorth [67] by using its neuron cores to emulate the learning; however, it incurs high overheads as additional neuron cores and connections are required to emulate the synaptic behavior. Third, Loihi [18], the state-of-the-art neuromorphic accelerator with on-chip learning support, has limited support toward diverse neuronal behaviors and various learning rules. It supports only a limited set of biological phenomena which can be expressed with a simple sum-of-products formula. For instance, it cannot simulate STP which greatly enriches a biological neural network’s dynamical behaviors [93], complex spike pairing methods such as suppression model, and the effect of neurons’ membrane potential on stabilization. Fourth, Flexon [53] is a flexible digital neuron capable of simulating the diverse neurons of the human brain. It achieves high flexibility in terms of neurons; however, it only supports the neuron computation stage of the brain simulation. Therefore, a flexible hardware accelerator supporting the diverse learning rules is a key requirement of brain simulations.

3.3 Design Goals

Motivated by the limited performance and flexibility of the existing accelerators, we need a new highly flexible and efficient accelerator achieving the following design goals. First, it should be flexible by supporting the diverse learning rules of the human brain. Second, it should achieve high efficiency in simulation speed and energy efficiency. Third, it should be easily applicable to the existing accelerators to realize the end-to-end brain simulation (e.g., digital neurons [53, 80]).

4 FLEXLEARN ARCHITECTURE

In this section we present *FlexLearn*, a flexible and highly efficient on-chip learning accelerator for brain simulations. We first perform an extensive analysis of prior neuroscience publications to figure

Table 2: Summary of the representative sub-rules for learning

Biological Role	Category	Sub-rule
Long-Term Plasticity (LT)	1. Hebbian (HB)	1.1. Nearest Pair [48]
		1.2. All-to-All Pair [28]
		1.3. Triplet Pair [70]
		1.4. Suppression Pair [25]
		1.5. Momentary Voltage [12]
		1.6. Average Voltage [17]
	2. Weight Dependence (WD)	2.1. Additive [84]
		2.2. Multiplicative [95]
		2.3. Sublinear [35]
		2.4. Mixed [97]
	3. Reward-Based (RB)	3.1. Three-Factor STDP [63]
Short-Term Plasticity (STP)	4. Fast Dynamics (FD)	4.1. Separate [4]
		4.2. Merged [92]
Homeostasis	5. Activity-Dependent Homeostasis (ADH)	5.1. Spike Dependent [97]
		5.2. Voltage Dependent [17]
	6. Activity-Independent Homeostasis (AIH)	6.1. Linear Decay [12]
		6.2. Exponential Decay [28]

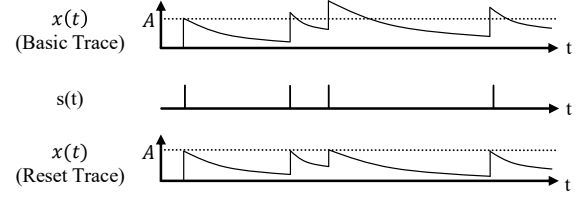
out various learning rules. With the analysis, we identify 17 representative sub-rules and classify them into six different categories. Next, we design per-category datapaths and use the datapaths as the building blocks of FlexLearn (Section 4.1). After designing the datapaths, we integrate and pipeline the datapaths according to their dependencies (Section 4.2). Then, we design and present an example brain simulation processor by integrating FlexLearn with the state-of-the-art flexible digital neuron to to perform end-to-end brain simulations (Section 4.3).

4.1 Target Learning (Sub-)Rules

To identify the target learning rules of FlexLearn, we perform an extensive analysis of prior neuroscience publications on the human brain [4, 12, 17, 25, 28, 35, 48, 63, 70, 84, 92, 95, 97]. The literature analysis indicates the critical need for supporting three major biological roles of learning (i.e., Long-Term Plasticity, Short-Term Plasticity, Homeostasis) whose detailed biological characteristics can be flexibly configured as a combination of sub-rules from the six categories. (i.e., Hebbian, Weight Dependence, Reward-Based, Fast Dynamics, Activity-Dependent Homeostasis, Activity-Independent Homeostasis)

Then, we observe that different learning rules share a set of common sub-rules from the categories and the flexible integrations of the sub-rules can implement various high-level functions of the brain [49]. Thus, implementing the sub-rules would be the most cost-effective way to implement the wide-spectrum of learning rules. More importantly, if a new learning rule is to be simulated in future, it can be effectively done by adding only unique sub-rules required by the new rule. In this way, the sub-rule implementation continues to enable flexible and efficient learning simulations. Table 2 summarizes our target learning rules and their sub-rules.

Next, to minimize the sub-rule implementation costs, we design a set of per-category datapaths. For the purpose, we first identify five types of traces which track the temporal history of the biological phenomena affecting the amount of weight changes in different sub-rules. After designing the datapaths for the traces, we design the

**Figure 4: Changes in traces with respect to spikes**

datapaths for how the sub-rules in different categories correlate the traces. By separating the datapath for the traces and the method for correlating them, we can reuse the trace datapath for different sub-rules. The datapaths are then integrated to achieve high flexibility. When integrating the datapaths, we minimize the hardware costs by eliminating the redundant sub-datapaths shared among the datapaths. In this section, we explain the mathematical models of the sub-rules in detail to show how we design and optimize the datapaths.

To facilitate mathematical descriptions of the learning rules, let us consider a synapse which transmits spikes from a pre-synaptic neuron i and a post-synaptic neuron j and the synapse's synaptic weight is w_{ij} . Then, assume that we are running a brain simulation for an interval of Δt whose current biological time is t . In such a condition, a learning rule of the synapse can be seen as a function which produces the synaptic state of the synapse (e.g., the value of w_{ij}) at time $t + \Delta t$ by taking as input 1) the synaptic state at time t , 2) whether the neurons fired a spike during $(t, t + \Delta t]$, and 3) the set of learning rules applied to the synapse. Using these mathematical definitions, we first describe and design datapaths for the traces that capture the various biological phenomena required in the 17 sub-rules (Section 4.1.1). Then, we describe how each sub-rule correlates the trace values to calculate the amount of weight change in the synapses and design the datapaths implementing them (Section 4.1.2–4.1.4).

4.1.1 Traces. In the human brain, both the prior and the current activities of a neuron can incur changes in synaptic weight. To track the prior activities and the previous values within a single variable, various learning rules utilize the concept of traces [64]. The most basic form of a trace, $x(t)$ ($x(t) \geq 0$), for a neuron is defined as $x(t) = (1 - \frac{\Delta t}{\tau}) \cdot x(t - \Delta t) + A \cdot s(t)$ where $s(t)$ denotes occurrence of a spike at time t and τ denotes the decay constant of the trace value. If the neuron fires the spike at time t , $s(t) = 1$; otherwise, $s(t) = 0$. Whenever the neuron fires a spike, it updates its trace value $x(t)$ by a constant A and decays its value over time in the absence of a spike. Such dynamics of a trace enable accumulation of spiking activity of the neuron within a single variable.

We find that the learning rules utilize five types of per-neuron traces: basic, reset, dependent, linear, and voltage traces. The mathematical definitions for the traces are:

$$x(t) = \begin{cases} (1 - \frac{\Delta t}{\tau}) \cdot x(t - \Delta t) + A \cdot s(t) & \text{basic trace} \\ (1 - \frac{\Delta t}{\tau}) \cdot x(t - \Delta t) \cdot \bar{s}(t) + A \cdot s(t) & \text{reset trace} \\ (1 - \frac{\Delta t}{\tau}) \cdot x(t - \Delta t) + e(t - \Delta t) \cdot s(t) & \text{dependent trace} \\ (a + x(t - \Delta t)) \cdot \bar{s}(t) & \text{linear trace} \\ (1 - \frac{\Delta t}{\tau}) \cdot x(t - \Delta t) + \frac{\Delta t}{\tau} \cdot v(t - \Delta t) & \text{voltage trace} \end{cases}$$

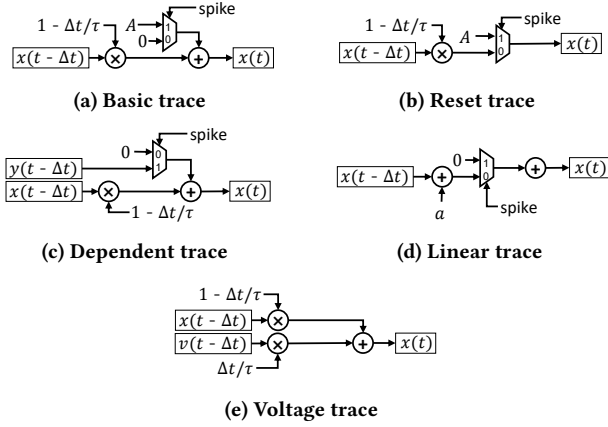


Figure 5: Datapaths for calculating traces

where the $\bar{s}(t) = 1$ in the absence of the spike and $\bar{s}(t) = 0$ if the neuron fires the spike. Also, e in the dependent trace represents an arbitrary trace (e.g., reset trace) that the trace depends on. Then, a in the linear trace is a pre-defined constant, and $v(t - \Delta t)$ in the voltage trace is the neuron's membrane potential at $t - \Delta t$. Figure 4 demonstrates how traces change with respect to the neuron's spiking activities. In the example, when a neuron fires a spike, the basic trace increases by A whereas the reset trace is set to A . In this way, the basic trace captures all previous spiking history; however, the reset trace tracks only the most recent spiking activity. Figures 5a–5e show the datapaths implementing the five traces; spike in the figures denotes whether the neuron fires a spike at the given timestep.

4.1.2 Long-Term Plasticity. Long-Term Plasticity (LT) is the first type of the biological roles we consider. Two forms of LT exist: Long-Term Depression (LTD) and Long-Term Potentiation (LTP) which weaken and strengthen inter-neuron connections, respectively [94]. Due to their long-lasting effects, LTD and LTP are thought to be highly relevant to memory [72]. The high importance of LT has attracted significant interests from neuroscientists ever since Hebb's postulate [37] was proposed.

Three biological phenomena take part in the LT: the interaction between the pre- and post- synaptic neurons, the synaptic weight, and the external stimulus. The sub-rules from the three categories (i.e., Hebbian (HB), Weight Dependence (WD), Reward-Based (RB)) describe each of the phenomena and are combined to take part in the LT. LT is modeled as:

$$w(t) = w(t - \Delta t) + \overbrace{RB(WD_+(t) \cdot HB_+(t) + WD_-(t) \cdot HB_-(t))}^{LTP} \quad (1)$$

where Δt denotes the timestep, and $w(t)$ is the synaptic weight at time t . $WD_+(t)$ and $WD_-(t)$ represent $WD(t)$ for LTP and LTD, respectively. The same applies to $HB_+(t)$ and $HB_-(t)$.

The STDP learning rule, for example, one of the most well-known methods for explaining the long term plasticity, consists of two sub-rule categories: HB and WD. Mathematically, the standard form

of STDP is defined as:

$$w(t) = w(t - \Delta t) + \overbrace{(w_{max} - w(t - \Delta t)) \cdot x_i(t) \cdot s_j(t)}^{LTP} - \underbrace{w(t - \Delta t) \cdot y_j(t) \cdot s_i(t)}_{LTD}$$

where w_{max} is a pre-defined largest weight value for the synapse and the term $w_{max} - w(t - \Delta t)$ and $w(t - \Delta t)$ represent $WD_+(t)$ and $WD_-(t)$. For the HB term, each neuron keeps two separate traces $x(t)$ and $y(t)$ for pre-synapse and post-synapse, respectively. In addition, the synaptic weight decreases (LTD) when the presynaptic neuron fires (i.e., $s_i(t) = 1$), and the synaptic weight increases (LTP) when the postsynaptic fires (i.e., $s_j(t) = 1$). The $x(t)$ from the pre-synaptic neuron (i.e., $x_i(t)$) and $y(t)$ from the post-synaptic neuron (i.e., $y_j(t)$) represent each case when multiplied with spikes of the post- and pre- synaptic neuron $s_j(t)$ and $s_i(t)$. According to the definition, various long-term plasticity models have different 1) model for explaining the HB term, 2) function for WD, and 3) existence of the RB term.

Hebbian Term: This category captures the interaction between the pre- and post- synaptic neurons in terms of their spiking activities and the membrane voltage of the post-synaptic neuron. Six different sub-rules fall into this category: nearest pair, all-to-all pair, triplet pair, suppression pair, momentary voltage, and average voltage. When updating the synaptic weight, the first four models consider various spike pairs between the pre- and post- neurons by utilizing different traces. On the other hand, the last two sub-rules account for the pre-synaptic neuron's activities and the membrane voltage of the post-synaptic neuron when changing the synaptic weight.

First, the nearest-neighbor pair model considers only the two latest spikes, one from the pre-synaptic neuron and the other from the post-synaptic neuron, using the reset trace. Second, the all-to-all pair model utilizes the basic trace to make a pre-synaptic spike and a post-synaptic spike interact with all prior post-synaptic spikes and all prior pre-synaptic spikes, respectively. The two sub-rules use the HB term defined as:

$$HB_+(t) = x_i(t) \cdot s_j(t), \quad HB_-(t) = y_j(t) \cdot s_i(t) \quad (2)$$

The two aforementioned sub-rules only pair a single post-synaptic spike with a single pre-synaptic spike. However, such single-spike pairs cannot describe the frequency effects on synaptic weight updates [70], requiring additional traces. To do so, the triplet and suppression sub-rules use:

$$HB_+(t) = x_{i1}(t) \cdot y_{j2}(t) \cdot s_j(t) \\ HB_-(t) = y_{j1}(t) \cdot x_{i2}(t) \cdot s_i(t)$$

where $x_{i1}(t)$ and $y_{j1}(t)$ correspond to $x_i(t)$ and $y_j(t)$ in Equation 2, and $x_{i2}(t)$ and $y_{j2}(t)$ are newly adopted traces to explain the effect of multiple spike pairs (e.g., pre-post-pre). Specifically, triplet sub-rule employs the basic traces as the two new traces:

$$x_{i1}(t) = (1 - \Delta t / \tau_{x1}) \cdot x_{i1}(t - \Delta t) + s_i(t) \\ x_{i2}(t) = (1 - \Delta t / \tau_{x2}) \cdot x_{i2}(t - \Delta t) + s_i(t)$$

where τ_{x1} and τ_{x2} denote the decay constants for the traces and $y_{j1}(t)$ and $y_{j2}(t)$ follow the same equation. On the other hand, suppression sub-rule defines $x_{i1}(t)$ and $y_{j1}(t)$ as the dependency

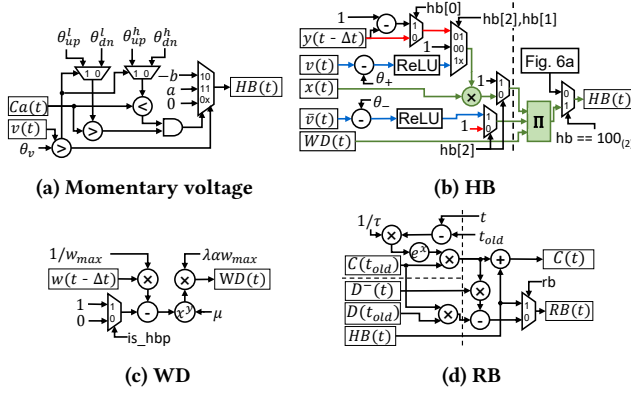


Figure 6: Datapaths for long-term plasticity

traces and $x_{i2}(t)$ and $y_{j2}(t)$ as the reset traces, resulting in:

$$\begin{aligned} x_{i1}(t) &= (1 - \Delta t / \tau_{x1}) \cdot x_{i1}(t - \Delta t) + s_i(t) \cdot x_{i2}(t - \Delta t) \\ x_{i2}(t) &= (1 - \Delta t / \tau_{x2}) \cdot x_{i2}(t - \Delta t) + \bar{s}_i(t) + s_i(t) \end{aligned}$$

as the dynamics of the traces.

Until now, we discussed the HB term whose synaptic weight updates depend on spike pairings and the current synaptic weights. However, prior neuroscience publications report that the updates can also depend on the membrane voltage of post-synaptic neurons [12, 17]. Two sub-rules model this phenomenon in different manner: momentary voltage [12] and average voltage [17].

The momentary voltage model determines the direction of the synaptic weight according to the instant voltage ($v(t)$) of the post-synaptic neuron. It introduces an additional basic trace (called calcium trace) $Ca_j(t)$ to determine if a synaptic weight update should be performed or not. The model is defined as:

$$\begin{aligned} Ca_j(t) &= (1 - \Delta t / \tau) \cdot Ca_j(t - \Delta t) + J_c \cdot s_j(t) \\ HB_+(t) &= a \cdot s_i(t) \quad \text{if } v(t) > \theta_v \text{ and } \theta_{up}^l < Ca_j(t) < \theta_{up}^h \\ HB_-(t) &= b \cdot s_i(t) \quad \text{if } v(t) \leq \theta_v \text{ and } \theta_{dn}^l < Ca_j(t) < \theta_{dn}^h \end{aligned}$$

where J_c is a pre-defined constant, $\theta_{\{up,dn\}}^{\{l,h\}}$ define the upper and the lower bounds of facilitation and depression, the a , and b is a pre-defined constant determining the amount of weight change when the condition is met. Figure 6a shows the datapath implementing the momentary voltage model; hb in the figure denotes the microcode in Table 3.

Second, the average voltage model adopts different form of voltage; the low pass filtered voltage determines the amount of synaptic weight updates. The mathematical definition of the average voltage model introduces a voltage trace $\bar{v}_{\pm}(t)$ and is defined as:

$$\begin{aligned} x_i(t) &= (1 - \Delta t / \tau_x) \cdot x_i(t - \Delta t) + s_i(t) \\ \bar{v}_-(t) &= (1 - \Delta t / \tau_-) \cdot \bar{v}_-(t - \Delta t) + \Delta t / \tau_- \cdot v(t - \Delta t) \\ \bar{v}_+(t) &= (1 - \Delta t / \tau_+) \cdot \bar{v}_+(t - \Delta t) + \Delta t / \tau_+ \cdot v(t - \Delta t) \\ HB_+(t) &= x_i(t) \cdot ReLU(\bar{v}_+(t) - \theta_-) \cdot ReLU(v(t) - \theta_+) \cdot s_j(t) \\ HB_-(t) &= ReLU(\bar{v}_-(t) - \theta_-) \cdot s_i(t) \end{aligned}$$

where $x_i(t)$ defines a basic trace and $ReLU(x) = \max(0, x)$. Also, θ_+ and θ_- define the lower bounds for voltage and voltage trace, respectively, and $\tau_{\pm}$ are pre-defined decay constants.

We reduce the hardware cost by making the datapaths share circuit for the common sub-datapaths. Figure 6b shows the integrated datapath for the HB term. The red line and the blue line in the figure represent the datapath for the triplet pair model and the average voltage model respectively. Also, the green line in the figure represents the common sub-expression shared among the two sub-rules. As shown in the figure, we reduce the integrated datapath by making the different sub-rules share the same circuit for the common sub-expression.

Weight Dependence: As discussed in Equation 1, the amount of synaptic weight changes due to LT depends on the definition of $WD_{\pm}(t)$. Weight dependence, the synaptic weight updates depending on the current synaptic weight, can lead to stability and biologically plausible distribution of synaptic weights [30]. From prior neuroscience publications, we find that four biologically plausible definitions for $WD_{\pm}(t)$ exist: additive [84], multiplicative [95], sublinear [35], and mixed [97] models. The mathematical definition is:

$$WD_{\pm}(t) = \begin{cases} \lambda, -\lambda\alpha & \text{additive} \\ \lambda w_{max}(1 - \frac{w}{w_{max}}), -\lambda\alpha w & \text{multiplicative} \\ \lambda w_{max}(1 - \frac{w}{w_{max}})^{\mu}, -\lambda\alpha w_{max}(\frac{w}{w_{max}})^{\mu} & \text{sublinear} \\ c_p, -c_d w & \text{mixed} \end{cases}$$

where the left and right formula indicates WD_+ and WD_- , respectively. λ denotes a learning rate, α is the ratio of LTP and LTD, μ is a constant, and c_p and c_d represent the average amounts of potentiation and relative depression, respectively, after one spike pairing. Among the models, additive model has no dependency on the current synaptic weight and imposes hard boundaries on synaptic weights. Figure 6c shows the datapath implementing the four $WD_{\pm}(t)$ models; is_hbp in the figure denotes whether the datapath is used for ltp or for ltd. Instead of using multiplexers with microcodes to support various types of sub-rules, we select specific parameters to emulate different sub-rules based on the microcode (e.g., $\mu = 1$ for multiplicative sub-rule).

Reward-Based STDP: The aforementioned sub-rules implement unsupervised learning; however, reinforcement learning can be described in a biologically plausible description [47]. Additional traces for the external stimulus track the rewards for eligible actions.

To augment the concept of reward to the learning rules, three-factor STDP model [47], an implementation of reward-based STDP, introduces two additional traces: dopamine (D_i) and eligibility (C_{ij}) traces represent low pass filtered version of the dopaminergic spikes and HB term outcomes ($HB_{\pm}(t)$). Mathematically, three-factor STDP is defined as:

$$\begin{aligned} C_{ij}^-(t) &= e^{-(t-t_{old})/\tau_c} \cdot C_{ij}(t_{old}) \\ C_{ij}(t) &= C_{ij}^-(t) + WD_{\pm}(t) \cdot HB_{\pm}(t) \cdot s_{i,j}(t) \\ D_j^-(t) &= (1 - \Delta t / \tau_d) \cdot D_j(t - \Delta t), \quad D_j(t) = D_j^-(t) + s_d(t) \\ RB(t) &= (C_{ij}^-(t) \cdot D_j^-(t) - C_{ij}(t_{old}) \cdot D_j(t_{old})) \cdot s_{i,j,d}(t) \\ t_{old} &= t \cdot s_{i,j,d}(t) + t_{old} \cdot \bar{s}_{i,j,d}(t) \end{aligned}$$

where the $s_d(t)$ is the incoming dopaminergic spikes and t_{old} is the latest spiking time of the pre-synaptic-, post-synaptic- or dopaminergic spikes. Figure 6d illustrates the datapath for three-factor STDP; rb in the figure denotes the microcode in Table 3.

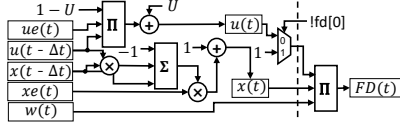


Figure 7: Datapath for short-term plasticity

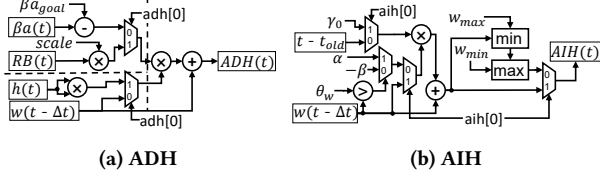


Figure 8: Datapaths for homeostasis

4.1.3 Short-Term Plasticity. Along with the LT, an inherent dynamics lasting 100 ms to about a second exists in biological synapses [33, 59]. The short-lasting dynamics is called Short-Term Plasticity (STP). STP has been shown to encode temporal information in biological neural networks by changing the effective synaptic weights according to inter-spike intervals [26]. From a computational point of view, it is plausible that STP may be an underlying mechanism of a number of processes which occur in daily life (e.g., speech recognition) [93].

Fast Dynamics: The Fast Dynamics (FD) category is responsible for explaining the STP: separate [4] and merged [92] model. First, the separate model is defined as:

$$P(t) = \frac{(P_0 + (\tau_P - 1) \cdot P(t - \Delta t))}{\tau_P} + f \cdot (1 - P(t - \Delta t)) \cdot s_i(t)$$

$$FD(t) = P(t) \cdot w(t)$$

where P_0 , f , and τ_P is a constant and $P(t)$ is a state variable tracking the spiking activities of the pre-synaptic neuron. Also, the result of $FD(t)$ denotes the effective synaptic weight, which lasts only temporally, at time t . Although the model is simple and easy to implement, it can model only either the facilitating or the depressing effect caused by a sequence of spikes; the two effects cannot be taken into account concurrently.

Merged model avoids the limitation by employing two state variables (i.e., $x(t)$ and $u(t)$) rather than one:

$$xe(t) = (1 - \Delta t / \tau_{rec}) \cdot xe(t - \Delta t) \cdot \bar{s}_i(t) + s_i(t)$$

$$ue(t) = (1 - \Delta t / \tau_{fac}) \cdot ue(t - \Delta t) \cdot \bar{s}_i(t) + s_i(t)$$

$$x(t) = 1 + (x(t - \Delta t)(1 - u(t - \Delta t)) - 1) \cdot xe(t - \Delta t)$$

$$u(t) = U + u(t - \Delta t) \cdot (1 - U) \cdot ue(t - \Delta t)$$

$$FD(t) = x(t) \cdot u(t) \cdot w(t) \cdot s_i(t)$$

where $xe(t)$ and $ue(t)$ denotes the temporal distance between the two latest pre-synaptic spikes with reset traces with pre-defined decay constant $\tau_{\{fac, rec\}}$. The state variables, $x(t)$ and $u(t)$ account for the depressing and the facilitating effects, respectively, of a series of spikes, and the two state variables are updated only when $s_i(t) = 1$. The variables are affected only when the pre-synaptic spikes occurs. For instance, if the frequency of the pre-synaptic spikes increases, the value of $x(t)$ decreases and reduces the effective

weight. The datapath implementing the two STP models is shown in Figure 7; fd in the figure denotes the microcode in Table 3.

4.1.4 Homeostasis. The last type of biological role synaptic plasticity affects is the Homeostasis; the ability of biological neural networks to modulate the activity of synapses [89, 96]. Homeostasis counter-balances the synaptic weight changes by LT in a way the synaptic weight distribution of an SNN does not deviate from a stable form. Depending on whether the activity of the neuron is relevant to the synaptic change, sub-rules are classified into two different categories: Activity-Dependent Homeostasis (ADH) and Activity-Independent Homeostasis (AIH) term. The general model for explaining the effect of homeostasis is as follows:

$$w(t) = ADH(t) + AIH(t)$$

where the $ADH(t)$ and $AIH(t)$ represent AIH term and ADH term respectively.

Activity-Dependent Homeostasis: Two sub-rules fall into this category: spike dependent [97] and voltage dependent [17] models. Spike dependent model changes the direction and the amount of synaptic weight update depending on the activity of the post-synaptic neuron. The model is defined as:

$$a(t) = (1 - \Delta t / \tau) \cdot a(t - \Delta t) + s_j(t) \cdot \Delta t / \tau$$

$$ADH(t) = \beta \cdot w(t - \Delta t) \cdot (a_{goal} - a(t))$$

where $a(t)$ is a basic trace to reflect the spiking activity. The β is a scaling constant reflecting the amount of weight changes induced by the sub-rule, and a_{goal} is a constant reflecting the desired value of the $a(t)$ value.

The other model, voltage dependent model, depresses the amount of synaptic weight updates by LT as follows:

$$h(t) = (1 - \Delta t / \tau) \cdot h(t - \Delta t) + \Delta t / \tau \cdot v(t - \Delta t) + s_i(t)$$

$$ADH(t) = \frac{h(t)^2}{scale} \cdot RB(t)$$

where $h(t)$ is a slight variation of the voltage trace, the average of the membrane potential and spiking activity of the post-synaptic neuron over a long timescale. Also, $scale$ is a scaling constant to reflect the amount of weight changes.

Activity-Independent Homeostasis: The two sub-rules in this category, linear decay [12] and exponential decay [28] models, do not take neuronal activity into account. First, linear decay model is defined as:

$$AIH(t) = \begin{cases} w(t - \Delta t) + \alpha(t - t_{old}) \cdot s_{i,j}(t) & \text{if } w(t - \Delta t) > \theta_w \\ w(t - \Delta t) - \beta(t - t_{old}) \cdot s_{i,j}(t) & \text{if } w(t - \Delta t) < \theta_w \end{cases}$$

$$t_{old} = t \cdot s_{i,j} + t_{old} \cdot \bar{s}_{i,j}$$

where α and β is a constant determining the amount of weight change by the sub-rule, and t_{old} refers to the latest spiking time of the pre-synaptic- and post-synaptic neuron. Depending on the current synaptic weight, the synaptic weight can saturate to extreme values. This model can stabilize synaptic weights against spontaneous activity. Second, exponential decay model emulates exponential decays by employing a decay constant γ_0 as follows:

$$AIH(t) = \gamma_0 \cdot w(t - \Delta t)$$

By doing so, the model can be used as a depression mechanism in the absence of other depression mechanisms [28]. Figure 8 shows the

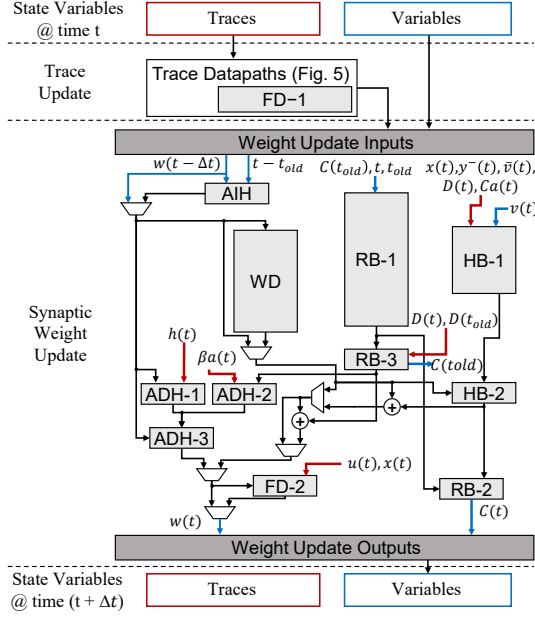


Figure 9: Datapath Integration in FlexLearn Architecture

Table 3: Microcodes to configure the datapaths

Signal	Description	Values
hbp[2:0] / hbd[2:0]	Select the model in Hebbian category for LTP/LTD	0: Nearest Pair 1: All-to-All Pair 2: Triplet Pair 3: Suppression Pair 4: Momentary Voltage 5: Average Voltage 6: Disable
wd[2:0]	Select the model in Weight Dependence category	0: Additive 1: Multiplicative 2: Sublinear 3: Mixed 4: Disable
rb	Enable or disable Reward-Based category	0: Enable 1: Disable
fd[1:0]	Select the model in Fast Dynamics category	0: Separate 1: Merged 2: Disable
adh[1:0]	Select the model in Activity-Dependent Homeostasis category	0: Spike Dependent 1: Voltage Dependent 2: Disable
aih[1:0]	Select the model in Activity-InDependent Homeostasis category	0: Linear Decay 1: Exponential decay 2: Disable

datapaths implementing these homeostatic plasticity mechanisms; adh and aih in the figure denotes the microcode in Table 3.

4.2 Efficient Integration of the Datapaths

We now present FlexLearn architecture, a flexible and highly efficient on-chip learning accelerator supporting the diverse learning rules. FlexLearn integrates the per-category datapaths to achieve

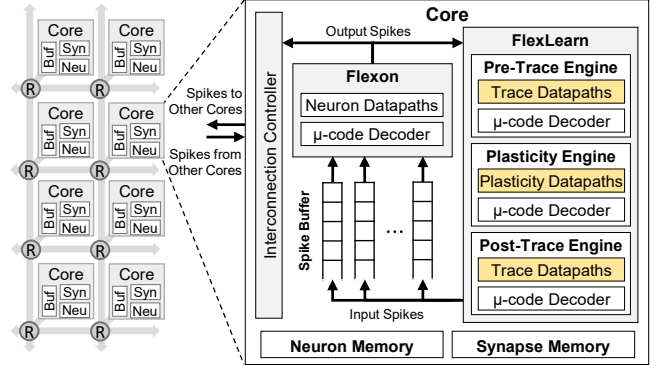


Figure 10: Example flexible brain simulation processor with FlexLearn

high flexibility. Figure 9 shows how FlexLearn integrates the datapaths. Given the current neuronal state (e.g., membrane voltage, neuron trace) and synaptic state (e.g., synaptic weight) variables at time t , the integrated datapath produces the updated state variables (e.g., neuron trace, synaptic weight) for the next timestep $t + \Delta t$ according to the specified learning rules.

Aimed at achieving higher performance, FlexLearn splits the datapaths into smaller sub-datapaths. For example, the ADT datapath is split into three sub-datapaths ADT- $\{1, 2, 3\}$. By doing so, FlexLearn can implement an 8-stage pipeline to improve throughput. Furthermore, employing the sub-datapaths allows data-parallel sub-datapaths to run in parallel. As an example, when sub-rules RB and HB are enabled, sub-datapaths RB-1 and HB-1 can run in parallel. Lastly, we identify some sub-rules that update weights for the synapses only when any of the connected neurons to the synapses fire spikes (e.g., HB). To improve efficiency, we perform weight update only for the synapses connected to neurons that fire spikes for such sub-rules.

As the datapaths should be enabled only when the target learning rule demands them, FlexLearn provides 16-bit microcodes to selectively activate the datapaths (Table 3). Using the datapaths, brain simulators can program FlexLearn to simulate their target learning rules. The microcodes instruct which of the datapaths should be activated; the input to the multiplexers within FlexLearn is determined by parsing the given microcodes. One 16-bit microcode corresponds to a learning rule for a synapse. By assigning different microcodes to synapses, FlexLearn facilitates highly flexible brain simulations involving diverse learning rules.

4.3 End-to-End Brain Simulation Processor

The datapaths of FlexLearn provide highly flexible learning; however, end-to-end brain simulations demand high flexibility in both learning and neurons. To enable the end-to-end simulations, we design and present an example brain simulation processor by integrating FlexLearn architecture on top of the scalable architecture similar to Loihi [18]. Loihi is the state-of-the-art brain simulation hardware which supports up to 128 neuromorphic cores per chip and a scalable asynchronous Network-on-Chip (NoC) for up to

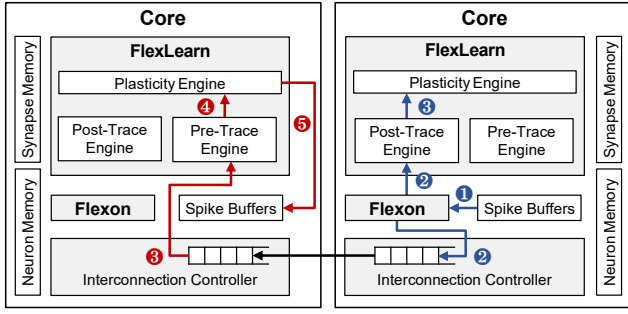


Figure 11: Working model of our example processor

16,348 chips. Despite the advantages in scalability, Loihi has limited support for both the learning rules and the neurons of the human brain; Loihi supports only 11 out of our target 17 sub-rules, and CUBA-variant neurons. To achieve high flexibility in neurons, our accelerator employs Flexon [53]. Flexon is the state-of-the-art digital neuron supporting the diverse neuron behaviors within the human brain; however, it only supports the neuron computation stage of the brain simulation. By integrating the datapaths with Flexon, our processor achieves highly flexible learning (using FlexLearn) and neurons (using Flexon) at the same time.

Figure 10 illustrates the architecture of our processor. The cores of the accelerator simulate on-chip learning and neuron computation, and are interconnected using the 2D mesh interconnection network of Loihi. Each core consists of the datapaths of FlexLearn and Flexon, spike buffers storing incoming spikes toward the core’s neurons, an interconnection controller, and neuron and synapse memory. The datapaths of FlexLearn and Flexon simulate on-chip learning and neurons, respectively. Each core can process up to 35 neurons and 28,000 synapses using a 0.23-MB per-core SRAM which serves as the neuron and the synapse memory. When a neuron of a source core fires a spike to a neuron of another destination core, the spike goes through the interconnection network to the destination core.

Figure 11 illustrates the working model of our processor. The processor simulates a timestep of a brain simulation as follows. First, Flexon retrieves input spikes from the spike buffers (❶), and then updates the target neuron’s internal state using the datapaths of Flexon. The information on whether the target neuron fires a spike gets propagated to the post-trace engine, and also to the destination neurons (possibly on other cores) if the neuron fires a spike (❷). After updating the neuron’s post-trace, a variable tracking the activities of the post-synaptic neuron, the plasticity engine receives the trace values and performs synaptic weight updates according to the synapse’s learning rules (❸). When the target neuron is on another core, a notification of the spike is sent to the destination core through the interconnection network. Then, the destination core’s pre-trace engine receives the notification as the spike is generated by the source/pre-synaptic neuron (❹ on the destination core). The pre-trace engine then updates the pre-trace value of the destination neuron and, applies the learning rules of the synapse, and the spike information is stored to the spike buffers of the destination core to trigger (❺–❻ on the destination core). By doing so, the destination

Table 4: RTL synthesis results of Loihi accelerator and FlexLearn

Accelerator	Component	Area [mm ²]	Power [mW]
128-Core Loihi	Neuron	0.4	35.3
	Learning	5.3	542.2
	SRAM	380.7	10360.2
	<i>Total</i>	386.4	10937.7
128-Core FlexLearn	Neuron	2.1	396.0
	Learning	27.7	3224.9
	SRAM	380.7	10360.2
	<i>Total</i>	410.5	13981.1

Table 5: Evaluated biological neural networks

Name	Neurons	Synapses	Compatibility				
			CPU	GPU	TrueNorth	Loihi	FlexLearn
Brunel [13]	5,750 CUB	Static: 1,190,250 Plastic: 2,116,000	✓	✓	✗	✓	✓
Vogels [98]	10,000 COBE	Static: 1,680,000 Plastic: 320,000	✓	✓	✗	✗	✓
Izhikevich [44]	1,000 Izhi.	Static: 360,000 Plastic: 640,000	✓	✓	✗	✗	✓

core initiates neuron computation of the destination neuron. In this way, our processor achieves not only highly flexible and efficient brain simulations, but also high scalability using the inter-core communication.

5 EVALUATION

5.1 Experimental Setup

To evaluate FlexLearn, we compare a 128-core brain simulation processor using FlexLearn against server-class CPU, GPU, and 128-core Loihi-style baseline processor. In this section, for brevity, we use the terms FlexLearn and Loihi to indicate the brain simulation processor using FlexLearn with Flexon and Loihi-style learning and neuron engine, respectively. 12-core Intel Xeon E5-2650 v4 CPU [1] and NVIDIA Titan X (Pascal) GPU [2] serve as the CPU and the GPU. The number of cores for the accelerators is chosen to be equal to that of Loihi [18]. The performance numbers for CPU and GPU were obtained from NEST [29] and GeNN [101] simulators, respectively. To measure the energy efficiency of the CPU and the GPU, we use Intel Performance Counter Monitor [100] and nvprof [3]. For Loihi accelerator and FlexLearn, we write Register-Transfer Level (RTL) code in Verilog, and synthesize the code using 45-nm NanGate FreePDK45 library [88]. The Static Random Access Memory (SRAM) costs are measured using CACTI 6.5 [65]. The synthesis result indicates that both accelerators operates in 250 MHz. Table 4 shows the synthesis results. Our FlexLearn accelerator greatly improves the flexibility over Loihi with only 6.3% chip area and 27.8% power overheads.

We use the biological neural networks used by Brunel [13], Vogels et al. [98], and Izhikevich [44] as the benchmarks. Table 5 summarizes their characteristics. Along with the biological neural networks, we utilize eight representative learning rule combinations (Table 6). The combinations are applied to the plastic synapses.

Table 6: Evaluated learning rule configurations

Type	Learning Rules	Compatibility				
		CPU	GPU	TrueNorth	Loihi	FlexLearn
A	1.5. Mementary	✓	✓	✗	✓	✓
	2.1. Additive	✓	✓	✗	✓	✓
	6.1. Linear Decay	✓	✓	✗	✓	✓
B	1.3. Triplet	✓	✓	✗	✓	✓
	2.2. Multiplicative	✓	✓	✗	✓	✓
	6.2. Exponential	✓	✓	✗	✓	✓
C	1.4. Suppression	✓	✓	✗	✗	✓
	2.2. Multiplicative	✓	✓	✗	✗	✓
D	1.2. All-to-All	✓	✓	✗	✗	✓
	2.2. Multiplicative	✓	✓	✗	✗	✓
	4.1. Separate	✓	✓	✗	✗	✓

Note that the underlying hardware may not support some configurations due to its inflexibility. For example, TrueNorth lacks support for all of the evaluated learning rule combinations, and Vogels and Izhikevich are not supported by Loihi due to its inflexibility in neuron model. On the other hand, FlexLearn is highly flexible and supports all of the configurations.

5.2 Flexible & Efficient Brain Simulations

In this experiment, we evaluate the brain simulation efficiency of our FlexLearn processor against the server-class CPU and GPU, and Loihi. We employ two performance metrics for the evaluation: simulation speed and energy consumption. For the comparison, we measure the latency and the energy consumption of the hardware when simulating the benchmarks for 10,000 timesteps. 10,000-timestep simulations emulate 1 s of biological time for Brunel whose timestep is 0.1 ms, and 10 s for Vogels and Izhikevich network as their timesteps are 1 ms.

Our results clearly indicate that FlexLearn significantly improves the brain simulation speed and energy efficiency. First, Figure 12 shows the speedups of our FlexLearn processor over the CPU and the Flexon-augmented CPU. FlexLearn achieves the harmonic mean speedups of 40.32 $\times$ over the CPU, and 27.44 $\times$ over the Flexon-augmented CPU. As Flexon targets only the neuron computation, the Flexon-augmented CPU fails to mitigate the high performance overheads of the learning rules. On the other hand, our FlexLearn processor greatly improves the performance by providing highly flexible learning rule support.

Second, Figure 13 shows the per-area performance improvements and the energy efficiency improvements of our FlexLearn processor over the other platforms. FlexLearn achieves harmonic mean performance improvement per area of 30.07 $\times$ and 37.12 $\times$ over the CPU and the GPU, respectively. In terms of the energy efficiency, FlexLearn achieves the harmonic mean improvements of 126.87 $\times$ and 262.16 $\times$ over the CPU and the GPU, respectively.

Moreover, FlexLearn outperforms Loihi despite improved flexibility by efficiently integrating the datapaths to minimize the hardware costs. Compared to Loihi, FlexLearn improves the performance per area and the energy efficiency by 1.41 $\times$ and 1.17 $\times$, respectively, in harmonic means. Note that the CPU outperforms the GPU as the brain simulations do not have enough parallelism to fully utilize

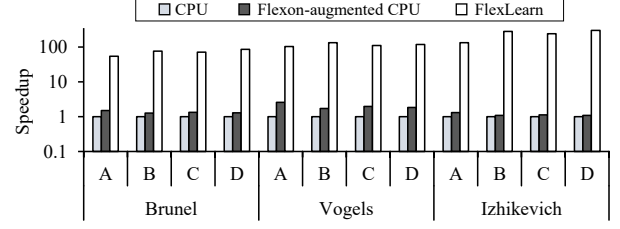
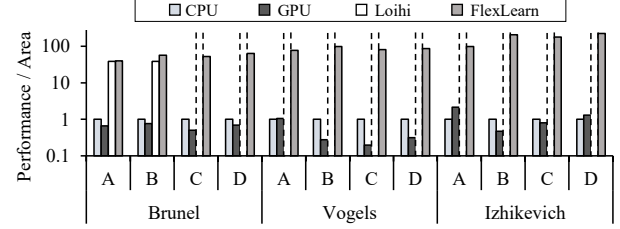
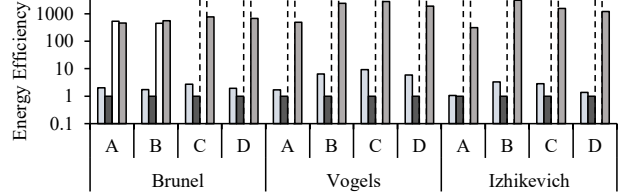
**Figure 12: Speedup of FlexLearn over the CPU****(a) Per-area performance improvements over the CPU****(b) Energy efficiency improvements over the GPU**

Figure 13: Improvements in the brain simulation efficiency of FlexLearn over the CPU, GPU, and Loihi. The dotted lines of Loihi indicate the simulations not supported by Loihi.

the GPU. Our analysis shows that the small synapse counts per neuron are the primary reason for the underutilization.

Our experimental results also indicate the high flexibility of FlexLearn over Loihi. In Figure 13, we observe that Loihi supports only two simulations. On the other hand, FlexLearn supports all of the simulations by implementing the 17 sub-rules; Loihi supports only 11 of the 17 sub-rules. Furthermore, by integrating Flexon with FlexLearn, our FlexLearn processor supports various neuron models as well.

5.3 Low Hardware Costs

Aimed at minimizing the hardware costs, our FlexLearn datapath exploits the common sub-expressions between the sub-rules of the same category. We further reduce the datapath by integrating the datapath for LTP and LTD. To evaluate the efficiency of the integration, we compare the hardware costs of FlexLearn’s integrated datapath to a naïve integration of the datapaths. The naïve integration simply places all of the datapaths and activates one of the datapaths at a time with respect to a given input. Figure 14 compares the chip area and the power consumption of each datapaths, the naïve integration, and FlexLearn’s integrated datapath. The results show that FlexLearn demands 27.8% smaller chip area and

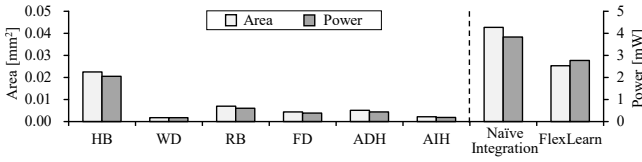


Figure 14: Chip area and power consumption of the per-rule datatypes and FlexLearn

consumes 40.8% less power than the naïve integration. By exploiting the characteristics of the learning rules, FlexLearn effectively reduces the hardware costs.

6 RELATED WORK

Brain Simulators and Accelerators: The properties of brain simulations can greatly differ in topology (e.g., numbers of neurons and synapses), neuronal behaviors (i.e., neuron models), and synapse properties (i.e., synaptic plasticity). To support a variety of brain simulations, general-purpose processors including CPUs and GPUs have been a favorable choice. Examples of CPU-based simulators include NEURON [38], GENESIS [11], NEST [29], Brian [32, 87], and Aurnyn [103]. GPU-based simulators (e.g., CARLsim [66], NeMo [21], NCS6 [41], GeNN [101]) provide faster simulation speeds by exploiting the parallelism inherent in neuronal dynamics. To improve simulation speeds, the simulators often utilize multiple machines to simulate large-scale biological neural networks. For instance, a simulation of 900 million neurons and 9 trillion synapses has been demonstrated by running C2 [7], a CPU-based cortical simulator developed by IBM Research, on top of 147,465 CPUs [6].

To improve the simulation speeds, some prior studies propose to employ FPGAs [16, 69, 85, 99]. Although FPGAs offer lower programmability than general-purpose processors do, FPGAs can be re-programmed to achieve high flexibility; users can re-program an FPGA-based accelerator to implement additional learning rules and neurons. NeuroFlow [16] and SNAVA [85] are example FPGA-based brain simulation accelerators. Unfortunately, the FPGA-based accelerators are limited by the amount of resources available on a target FPGA board. They also achieve lower performance than custom accelerators implementing the same functionalities [52].

Aimed at improving the simulation efficiency, custom-designed accelerators (e.g., TrueNorth [5, 62, 76], ROLLS [75], Loihi [18], ODIN [22–24]) employ digital circuits implementing the learning rules and the neurons of the human brain. These accelerators achieve fast simulation speeds and higher energy efficiency over general-purpose processors and FPGAs. For instance, Loihi [18] implements a limited set of learning rules which can be defined in a sum-of-products form taking as input spike counts and per-neuron traces. Some prior studies, aimed at achieving even higher energy efficiency, propose to employ analog circuits. Example analog-based accelerators include Neurogrid [9] and FACETS/BrainScaleS project [77–79].

Despite the high efficiency, the existing accelerators are inflexible; they support only a limited set of learning rules and neurons. To achieve high flexibility, SpiNNaker [27, 68, 71] couples custom-designed memory and communication architectures with low-power ARM968 CPUs which simulate neuronal and synaptic

dynamics. However, as the low-power CPUs simulate the learning rules and the neurons, SpiNNaker suffers from the low performance of the CPUs.

We focus on the learning rules of the human brain in this work; however, acceleration of the neurons is also an important topic as neuronal state updates always occur at each timestep. Examples of digital neurons, the custom-designed circuits specializing in simulating neurons, include the work by Smith [80] and Flexon [53]. Aimed at implementing a complete brain simulation accelerator, we demonstrated a Flexon-integrated FlexLearn prototype.

Brain Simulation Infrastructures: Brain simulations using the underlying hardware require software front-ends interfacing with the hardware. A typical workflow of a brain simulation is as follows. First, researchers specify the properties of their target biological neural networks (e.g., topology) using a hardware-independent description language such as PyNN [19] and NeuroML [14, 31]. Then, given the description, a neural network compiler (e.g., LCompiler [54, 55], a neuromorphic compiler by Ji et al. [50]) compiles the description and produces executable code for the underlying hardware or instruction set architecture (e.g., NISA [36]). During the compilation, the compiler may apply optimizations to improve performance; for instance, LCompiler for Intel Loihi optimizes neuron-to-core mapping to minimize energy consumption. After that, the produced executable runs on the underlying hardware. When the simulation outcomes are validated, the researcher can upload their brain simulation configurations to ModelDB [39], the open database for sharing latest findings in neuronal and synaptic dynamics.

Computing Like the Human Brain: Depending on how information is encoded, the abstraction of the human brain can be classified into two types: Rate-based Neural Networks (RNNs) and Temporal Neural Networks (TNNs) [81, 82]. Our work focuses on RNNs which use sequences of spikes from a *single* synapse to encode and convey information [12, 17, 20]. As RNNs use multiple spikes from a single synapse to communicate and compute, the highly complex interactions between the spikes need to be considered (e.g., homeostasis [104]). On the other hand, TNNs encode information in the spatio-temporal patterns between the spikes from *multiple* synapses [56, 91]. With TNNs, the complex interactions needed by RNNs are no longer necessary, allowing much more simpler abstractions to be used [10]. Using the space-time computing capability of TNNs, prior studies demonstrate that various brain-like computations are possible with TNNs [51, 57, 61, 73]. For instance, a recent study by Smith [83] utilizes TNNs to implement a generalized race logic. As RNNs and TNNs both demand learning rules to adjust the synaptic weights, FlexLearn can equally benefit both RNNs and TNNs. Note that both RNNs and TNNs are active research topics, and there is yet no consensus on which is more suitable to describe the computations of the brain.

7 CONCLUSION

We presented FlexLearn, a highly flexible and efficient on-chip learning architecture for brain simulations. Motivated by the low flexibility of the existing accelerators, FlexLearn achieves highly flexible on-chip learning by supporting diverse learning rules. FlexLearn achieves high flexibility by using the datatypes implementing the

sub-rules as the basic building blocks, and high efficiency by integrating the datapaths in an efficient manner. We then integrated FlexLearn with Flexon, the state-of-the-art digital neuron, to implement an end-to-end brain simulation accelerator. The resulting FlexLearn accelerator realizes fast and highly efficient brain simulations by achieving high flexibility.

ACKNOWLEDGEMENT

This work was partly supported by Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Science and ICT (MSIT) (NRF-2015M3C4A7065647, NRF2017R1A2B3011038, NRF-2019-Global Ph.D. Fellowship Program), Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.1711080972), and Creative Pioneering Researchers Program through Seoul National University. We also appreciate the support from Samsung Advanced Institute of Technology (SAIT), and Automation and Systems Research Institute (ASRI), Inter-university Semiconductor Research Center (ISRC) at Seoul National University.

REFERENCES

- [1] [n. d.]. Intel® Xeon® Processor E5-2650 v4. <https://ark.intel.com/products/91767/Intel-Xeon-Processor-E5-2650-v4-30M-Cache-2-20-GHz->.
- [2] [n. d.]. NVIDIA TITAN X Graphics Card for VR Gaming | NVIDIA GeForce. <https://www.nvidia.com/en-us/geforce/products/10series/titan-x-pascal/>.
- [3] [n. d.]. Profiler User's Guide. <https://docs.nvidia.com/cuda/profiler-users-guide/index.html>.
- [4] L. F. Abbott, J. A. Varela, Kamal Sen, and S. B. Nelson. 1997. Synaptic Depression and Cortical Gain Control. *Science* 275 (1997).
- [5] Filipp Akopyan, Jun Sawada, Andrew Cassidy, Rodrigo Alvarez-Icaza, John Arthur, Paul Merolla, Nabil Imam, Yutaka Nakamura, Pallab Datta, Gi-Joon Nam, Brian Taba, Michael Beakes, Bernard Brezzo, Jente B. Kuang, Rajit Manohar, William P. Risk, Bryan Jackson, and Dharmendra S. Modha. 2015. TrueNorth: Design and Tool Flow of a 65 mW 1 Million Neuron Programmable Neurosynaptic Chip. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 34, 10 (2015).
- [6] Rajagopal Ananthanarayanan, Steven K. Esser, Horst D. Simon, and Dharmendra S. Modha. 2009. The Cat is Out of the Bag: Cortical Simulations with 10^9 Neurons, 10^{13} Synapses. In *Proc. ACM/IEEE Conference on High Performance Networking and Computing (SC)*.
- [7] Rajagopal Ananthanarayanan and Dharmendra S. Modha. 2007. Anatomy of a Cortical Simulator. In *Proc. ACM/IEEE Conference on High Performance Networking and Computing (SC)*.
- [8] Frederico A.C. Azevedo, Ludmila R.B. Carvalho, Lea T. Grinberg, José Marcelo Farfel, Renata E.L. Ferretti, Renata E.P. Leite, Wilson Jacob Filho, and Suzana Herculano-Houzel. 2009. Equal numbers of neuronal and nonneuronal cells make the human brain an isometrically scaled-up primate brain. *The Journal of Comparative Neurology* 513, 5 (2009).
- [9] Ben Varkey Benjamin, Peiran Gao, Emmett McQuinn, Swadesh Choudhary, Anand R. Chandrasekaran, Jean-Marie Bussat, Rodrigo Alvarez-Icaza, John V. Arthur, Paul A. Merolla, and Kwabena Boahen. 2014. Neurogrid: A Mixed-Analog-Digital Multichip System for Large-Scale Neural Simulations. *Proc. IEEE* 102, 5 (2014).
- [10] Olivier Bichler, Damien Querlioz, Simon J. Thorpe, Jean-Philippe Bourgoin, and Christian Gamrat. 2012. Extraction of temporally correlated features from dynamic vision sensors with spike-timing-dependent plasticity. *Neural Networks* 32 (2012).
- [11] James M. Bower and David Beeman. 1998. *The Book of GENESIS: Exploring Realistic Neural Models with the General NEural Simulation System* (2 ed.). Springer-Verlag New York.
- [12] Joseph M. Brader, Walter Senn, and Stefano Fusi. 2007. Learning Real-World Stimuli in a Neural Network with Spike-Driven Synaptic Dynamics. *Neural Computation* 19 (2007).
- [13] Nicolas Brunel. 2000. Dynamics of Sparsely Connected Networks of Excitatory and Inhibitory Spiking Neurons. *Journal of Computational Neuroscience* 8, 3 (2000).
- [14] Robert C. Cannon, Pdraig Gleeson, Sharon Crook, Gautham Ganapathy, Boris Marin, Eugenio Piasini, and R. Angus Silver. 2014. LEMS: a language for expressing complex biological models in concise and hierarchical form and its use in underpinning NeuroML 2. *Frontiers in Neuroinformatics* 8 (2014).
- [15] Yongqiang Cao, Yang Chen, and Deepak Khosla. 2015. Spiking Deep Convolutional Neural Networks for Energy-Efficient Object Recognition. *International Journal of Computer Vision* 113, 1 (2015).
- [16] Kit Cheung, Simon R. Schultz, and Wayne Luk. 2016. NeuroFlow: A General Purpose Spiking Neural Network Simulation Platform using Customizable Processors. *Frontiers in Neuroscience* 9 (2016).
- [17] Claudia Clopath and Wulfram Gerstner. 2010. Voltage and Spike Timing Interact in STDP – A Unified Model. *Frontiers in Synaptic Neuroscience* 2, 25 (2010).
- [18] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, Yuyun Liao, Chit-Kwan Lin, Andrew Lines, Ruokun Liu, Deepak Mathaikutty, Steve McCoy, Arnab Paul, Jonathan Tse, Guruguhannathan, Venkataramanan, Yi-Hsin Weng, Andreas Wild, Yoonseok Yang, and Hong Wang. 2018. Loihi: A Neuromorphic Manycore Processor with On-Chip Learning. *IEEE Micro* 38, 1 (2018).
- [19] Andrew P. Davison, Daniel Brüderle, Jochen Eppler, Jens Kremkow, Eilif Müller, Dejan Pecevski, Laurent Perrinet, and Pierre Yger. 2009. PyNN: a common interface for neuronal network simulators. *Frontiers in Neuroinformatics* 2, 11 (2009).
- [20] Peter U. Diehl and Matthew Cook. 2015. Unsupervised learning of digit recognition using spike-timing-dependent plasticity. *Frontiers in Computational Neuroscience* 9 (2015).
- [21] Andreas K. Fidjeland and Murray P. Shanahan. 2010. Accelerated Simulation of Spiking Neural Networks Using GPUs. In *Proc. 2010 International Joint Conference on Neural Networks (IJCNN)*.
- [22] Charlotte Frenkel, Giacomo Indiveri, Jean-Didier Legat, and David Bol. 2017. A Fully-Synthesized 20-Gate Digital Spike-Based Synapse with Embedded Online Learning. In *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*.
- [23] Charlotte Frenkel, Martin Lefebvre, Jean-Didier Legat, and David Bol. 2018. A 0.086-mm^2 12.7-pJ/SOP 64k-Synapse 256-Neuron Online-Learning Digital Spiking Neuromorphic Processor in 28nm CMOS. *IEEE Transactions on Biomedical Circuits and Systems* (2018).
- [24] Charlotte Frenkel, Jean-Didier Legat, and David Bol. 2017. A Compact Phenomenological Digital Neuron Implementing the 20 Izhikevich Behaviors. In *Proc. IEEE Biomedical Circuits and Systems Conference (BioCAS)*.
- [25] Robert C. Froemke and Yang Dan. 2002. Spike-timing-dependent synaptic modification induced by natural spike trains. *Nature* 416 (2002).
- [26] Galit Fuhrmann, Idan Segev, Henry Markram, and Mishka Tsodyks. 2002. Coding of Temporal Information by Activity-Dependent Synapses. *Journal of Neurophysiology* 87 (2002).
- [27] Steve B. Furber, David R. Lester, Luis A. Plana, Jim D. Garside, Eustace Painkras, Steve Temple, and Andrew D. Brown. 2013. Overview of the SpiNNaker System Architecture. *IEEE Trans. Comput.* 62, 12 (2013).
- [28] Wulfram Gerstner and Werner M. Kistler. 2002. *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press.
- [29] Marc-Oliver Gewaltig and Markus Diesmann. 2007. NEST (NEural Simulation Tool). *Scholarpedia* 2, 4 (2007).
- [30] Matthieu Gilson and Tomoki Fukai. 2011. Stability versus Neuronal Specialization for STDP: Long-Tail Weight Distributions Solve the Dilemma. *PLoS One* 6, 10 (2011).
- [31] Pdraig Gleeson, Sharon Crook, Robert C. Cannon, Michael L. Hines, Guy O. Billings, Matteo Farinella, Thomas M. Morse, Andrew P. Davison, Subhasis Ray, Upinder S. Bhalla, Simon R. Barnes, Yoana D. Dimitrova, and R. Angus Silver. 2010. NeuroML: A Language for Describing Data Driven Models of Neurons and Networks with a High Degree of Biological Detail. *PLoS Computational Biology* 6, 6 (2010).
- [32] Dan F. M. Goodman and Romain Brette. 2009. The Brian simulator. *Frontiers in Neuroscience* 3, 2 (2009).
- [33] Anirudh Gupta, Yun Wang, and Henry Markram. 2000. Organizing Principles for a Diversity of GABAergic Interneurons and Synapses in the Neocortex. *Science* 287 (2000).
- [34] Wulfram Gerstner. 2014. *Neuronal Dynamics: From Single Neurons To Networks And Models Of Cognition*. Cambridge University Press.
- [35] R. Gütiğ, R. Aharonov, S. Rotter, and Haim Sompolinsky. 2003. Learning Input Correlations through Nonlinear Temporally Asymmetric Hebbian Plasticity. *Journal of Neuroscience* 23, 9 (2003).
- [36] Atif Hashmi, Andrew Nere, James Jamal Thomas, and Mikko Lipasti. 2011. A Case for Neuromorphic ISAs. In *Proc. 16th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [37] Donald O. Hebb. 1949. *The Organization of Behavior*. Wiley & Sons.
- [38] M.L. Hines and N.T. Carnevale. 1997. The NEURON Simulation Environment. *Neural Computation* 9 (1997).

- [39] Michael L. Hines, Thomas Morse, Michele Migliore, Nicholas T. Carnevale, and Gordon M. Shepherd. 2004. ModelDB: a Database to Support Computational Neuroscience. *Journal of Computational Neuroscience* 17, 1 (2004).
- [40] Naoki Hiratani and Tomoki Fukui. 2014. Interplay between short- and long-term plasticity in cell-assembly formation. *PLoS one* 9, 7 (2014), e101535.
- [41] Roger V. Hoang, Devyani Tanna, Laurence C. Jayet Bray, Sergiu M. Dascalu, and Frederick C. Harris Jr. 2013. A novel CPU/GPU simulation environment for large-scale biologically realistic neural modeling. *Frontiers in Neuroinformatics* 7, 19 (2013).
- [42] A. L. Hodgkin and A. F. Huxley. 1952. A Quantitative Description of Membrane Current and Its Application to Conduction and Excitation in Nerve. *Journal of Physiology* 117 (1952).
- [43] John R. Hughes. 1958. Post-Tetanic Potentiation. *Physiological Reviews* 38, 1 (1958).
- [44] Eugene M. Izhikevich. 2003. Simple Model of Spiking Neurons. *IEEE Transactions on Neural Networks* 14, 6 (2003).
- [45] Eugene M. Izhikevich. 2004. Which Model to Use for Cortical Spiking Neurons? *IEEE Transactions on Neural Networks* 15, 5 (2004).
- [46] Eugene M. Izhikevich. 2006. Polychronization: Computation with Spikes. *Neural Computation* 18 (2006).
- [47] Eugene M. Izhikevich. 2007. Solving the distal reward problem through linkage of STDP and dopamine signaling. *Cerebral cortex* 17, 10 (2007), 2443–2452.
- [48] Eugene M. Izhikevich and Niraj S. Desai. 2003. Relating STDP to BCM. *Neural Computation* 15 (2003).
- [49] Eugene M. Izhikevich and Gerald M. Edelman. 2008. Large-scale model of mammalian thalamocortical systems. *Proceedings of the National Academy of Sciences of the United States of America (PNAS)* 105, 9 (2008).
- [50] Yu Ji, Youhui Zhang, Wenguang Chen, and Yuan Xie. 2018. Bridging the Gap Between Neural Networks and Neuromorphic Hardware with A Neural Network Compiler. In *Proc. 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [51] Saeed Reza Kheradpisheh, Mohammad Ganjtabesh, and Timothée Masquelier. 2016. Bio-inspired unsupervised learning of visual features leads to robust invariant object recognition. *Neurocomputing* 205 (2016).
- [52] Ian Kuon and Jonathan Rose. 2007. Measuring the Gap Between FPGAs and ASICs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 26, 2 (2007).
- [53] Dayeol Lee, Gwangmu Lee, Dongup Kwon, Sunghwa Lee, Youngsok Kim, and Jangwoo Kim. 2018. Flexon: A Flexible Digital Neuron for Efficient Spiking Neural Network Simulations. In *Proc. 45th ACM/IEEE International Symposium on Computer Architecture (ISCA)*.
- [54] Chit-Kwan Lin, Andreas Wild, Gautham N. Chinya, Yongqiang Cao, Mike Davies, Daniel M. Lavery, and Hong Wang. 2018. Programming Spiking Neural Networks on Intel's Loihi. *Computer* 51, 3 (2018).
- [55] Chit-Kwan Lin, Andreas Wild, Gautham N. Chinya, Tsung-Han Lin, Mike Davies, and Hong Wang. 2018. Mapping Spiking Neural Networks onto a Manycore Neuromorphic Hardware. In *Proc. 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
- [56] Wolfgang Maass. 1997. Noisy Spiking Neurons with Temporal Coding have more Computational Power than Sigmoidal Neurons. *Advances in Neural Information Processing Systems* 9 (1997).
- [57] Wolfgang Maass, Thomas Natschläger, and Henry Markram. 2002. Real-Time Computing Without Stable States: A New Framework for Neural Computation Based on Perturbations. *Neural Computation* 14, 11 (2002).
- [58] Henry Markram, Joachim Lübke, Michael Frotscher, and Bert Sakmann. 1997. Regulation of synaptic efficacy by coincidence of postsynaptic APs and EPSPs. *Science* 275, 5297 (1997), 213–215.
- [59] Henry Markram, Yun Wang, and Misha Tsodyks. 1998. Differential signaling via the same axon of neocortical pyramidal neurons. *Proceedings of the National Academy of Sciences of the United States of America (PNAS)* 95, 9 (1998).
- [60] S. J. Martin, P. D. Grimwood, and R. G. M. Morris. 2000. Synaptic Plasticity and Memory: An Evaluation of the Hypothesis. *Annual Review of Neuroscience* 23 (2000).
- [61] Timothée Masquelier and Simon J. Thorpe. 2007. Unsupervised Learning of Visual Features through Spike Timing Dependent Plasticity. *PLoS Computational Biology* 3, 2 (2007).
- [62] Paul A. Merolla, John V. Arthur, Rodrigo Alvarez-Icaza, Andrew S. Cassidy, Jun Sawada, Filipp Akopyan, Bryan L. Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, Bernard Brezzo, Ivan Vo, Steven K. Esser, Rathinakumar Appuswamy, Brian Taba, Arnon Amir, Myron D. Flickner, William P. Risk, Rajit Manohar, and Dharmendra S. Modha. 2014. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science* 345, 6197 (2014).
- [63] Mantas Mikaitis, Garibaldi Pineda Garcia, James C. Knight, and Steve B. Furber. 2018. Neuromodulated Synaptic Plasticity on the SpiNNaker Neuromorphic System. *Frontiers in Neuroscience* 27 (2018).
- [64] Abigail Morrison, Markus Diesmann, and Wulfram Gerstner. 2008. Phenomenological models of synaptic plasticity based on spike timing. *Biological Cybernetics* 98, 6 (2008).
- [65] Naveen Muralimanohar, Rajeev Balasubramanian, and Norman P. Jouppi. 2009. CACTI 6.0: A tool to understand large caches. *University of Utah and Hewlett Packard Laboratories, Tech. Rep* 147 (2009).
- [66] Jayram Moorkanikara Nageswaran, Nikil Dutt, Jeffrey L. Krichmar, Alex Nicolau, and Alexander V. Veidenbaum. 2009. A configurable simulation environment for the efficient simulation of large-scale spiking neural networks on graphics processors. *Neural Networks* 22 (2009).
- [67] Andrew Nere, Atif Hashmi, Mikko Lipasti, and Giulio Tononi. 2013. Bridging the Semantic Gap: Emulating Biological Neuronal Behaviors with Simple Digital Neurons. In *Proc. 19th IEEE International Symposium on High Performance Computer Architecture (HPCA)*.
- [68] Eustace Painkras, Luis A. Plana, Jim Garside, Steve Temple, Simon Davidson, Jeffrey Pepper, David Clark, Cameron Patterson, and Steve Furber. 2012. SpiNNaker: A Multi-Core System-on-Chip for Massively-Parallel Neural Net Simulation. In *Proc. IEEE Custom Integrated Circuits Conference (CICC)*.
- [69] Danilo Pani, Paolo Meloni, Giuseppe Tuveri, Francesca Palumbo, Paolo Masobrio, and Luigi Raffo. 2017. An FPGA Platform for Real-Time Simulation of Spiking Neuronal Networks. *Frontiers in Neuroscience* 11 (2017).
- [70] Jean-Pascal Pfister and Wulfram Gerstner. 2006. Beyond Pair-Based STDP: a Phenomenological Rule for Spike Triplet and Frequency Effects. In *Advances in Neural Information Processing Systems* 18.
- [71] Luis A. Plana, David Clark, Simon Davidson, Steve Furber, Jim Garside, Eustace Painkras, Jeffrey Pepper, Steve Temple, and John Bainbridge. 2011. SpiNNaker: Design and Implementation of a GALS Multicore System-on-Chip. *ACM Journal on Emerging Technologies in Computing Systems* 7, 4 (2011).
- [72] Enrico Prati. 2016. Atomic scale nanoelectronics for quantum neuromorphic devices: comparing different materials. *International Journal of Nanotechnology* 13, 7 (2016).
- [73] Dimitri Probst, Wolfgang Maass, Henry Markram, and Marc-Oliver Gewaltig. 2012. Liquid Computing in a Simplified Model of Cortical Layer IV: Learning to Balance a Ball. In *Proc. 22nd International Conference on Artificial Neural Networks (ICANN)*.
- [74] Guo qiang Bi and Mu ming Poo. 2001. Synaptic Modification by Correlated Activity: Hebb's Postulate Revisited. *Annual Review of Neuroscience* 24 (2001).
- [75] Ning Qiao, Hesham Mostafa, Federico Corradi, Marc Osswald, Fabio Stefanini, Dora Sumislawska, and Giacomo Indiveri. 2015. A reconfigurable on-line learning spiking neuromorphic processor comprising 256 neurons and 128K synapses. *Frontiers in Neuroscience* 9 (2015).
- [76] Jun Sawada, Filipp Akopyan, Andrew S. Cassidy, Brian Taba, Michael V. Debole, Pallab Datta, Rodrigo Alvarez-Icaza, Arnon Amir, John V. Arthur, Alexander Andreopoulos, Rathinakumar Appuswamy, Heinz Baier, Davis Barch, David J. Berg, Carmelo di Nolfo, Steven K. Esser, Myron Flickner, Thomas A. Horvath, Bryan L. Jackson, Jeff Kusnitz, Scott Lekuch, Michael Mastro, Timothy Melano, Paul A. Merolla, Steven E. Millman, Tapan K. Nayak, Norm Pass, Hartmut E. Penner, William P. Risk, Kai Schleupen, Benjamin Shaw, Hayley Wu, Brian Giera, Adam T. Moody, Nathan Mundhenk, Brian C. Van Essen, Eric X. Wang, David P. Widemann, Qing Wu, William E. Murphy, Jamie K. Infantolino, James A. Ross, Dale R. Shires, Manuel M. Vindiola, Raju Namburu, and Dharmendra S. Modha. 2016. TrueNorth Ecosystem for Brain-Inspired Computing: Scalable Systems, Software, and Applications. In *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [77] Johannes Schemmel, Daniel Brüderle, Andreas Grübl, Matthias Hock, Karlheinz Meier, and Sebastian Millner. 2010. A Wafer-Scale Neuromorphic Hardware System for Large-Scale Neural Modeling. In *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*.
- [78] Johannes Schemmel, Johannes Fierens, and Karlheinz Meier. 2008. Wafer-scale Integration of Analog Neural Networks. In *Proc. IEEE International Joint Conference on Neural Networks (IJCNN)*.
- [79] Johannes Schemmel, Andreas Grübl, Karlheinz Meier, and Eilif Mueller. 2006. Implementing Synaptic Plasticity in a VLSI Spiking Neural Network Model. In *Proc. IEEE International Joint Conference on Neural Networks (IJCNN)*.
- [80] James E. Smith. 2014. Efficient Digital Neurons for Large Scale Cortical Architectures. In *Proc. 41st ACM/IEEE International Symposium on Computer Architecture (ISCA)*.
- [81] James E. Smith. 2017. Research Agenda: Spacetime Computation and the Neocortex. *IEEE Micro* 37, 1 (2017).
- [82] James E. Smith. 2017. *Space-Time Computing with Temporal Neural Networks*. Morgan & Claypool Publishers.
- [83] James E. Smith. 2018. Space-Time Algebra: A Model for Neocortical Computation. In *Proc. 45th ACM/IEEE International Symposium on Computer Architecture (ISCA)*.
- [84] Sen Song, Kenneth D. Miller, and L. F. Abbott. 2000. Competitive Hebbian learning through spike-timing-dependent synaptic plasticity. *Nature Neuroscience* 3 (2000).
- [85] Athul Sripad, Giovanni Sanchez, Mireya Zapata, Vito Pirrone, Taho Dorta, Salvatore Cambria, Albert Marti, Karthikeyan Krishnamourthy, and Jordi Madrenas. 2018. SNAVA—A real-time multi-FPGA multi-model spiking neural network simulation architecture. *Neural Networks* 97 (2018).

- [86] Richard B. Stein. 1965. A Theoretical Analysis of Neuronal Variability. *Biophysical Journal* 5, 2 (1965).
- [87] Marcel Stimberg, Dan F. M. Goodman, Victor Benichoux, and Romain Brette. 2014. Equation-oriented specification of neural models for simulations. *Frontiers in Neuroscience* 8, 6 (2014).
- [88] James E. Stine, Ivan Castellanos, Michael Wood, Fejj Henson, Fred Love, W. Rhett Davis, Paul D. Franzon, Michael Bucher, Sunil Basavarajiah, Julie Oh, and Ravi Jenkal. 2007. FreePDK: An Open-Source Variation-Aware Design Kit. In *Proc. IEEE International Conference on Microelectronic Systems Education (MSE)*.
- [89] D. James Surmeier and Robert Fehring. 2004. A mechanism for homeostatic plasticity. *Nature Neuroscience* 7 (2004).
- [90] Amirhossein Tavaneai and Anthony S. Maida. 2017. A spiking network that learns to extract spike signatures from speech signals. *Neurocomputing* 240 (2017).
- [91] Simon J. Thorpe and Michel Imbert. 1989. Biological Constraints on Connectionist Modelling. *Connectionism in Perspective* (1989).
- [92] Misha Tsodyks, Asher Uziel, and Henry Markram. 2000. Synchrony Generation in Recurrent Networks with Frequency-Dependent Synapses. *Journal of Neuroscience* 20 (2000).
- [93] Misha Tsodyks and Si Wu. 2013. Short-term synaptic plasticity. *Scholarpedia* 8, 10 (2013).
- [94] Tadaharu Tsumoto. 1992. Long-term potentiation and long-term depression in the neocortex. *Progress in neurobiology* 39, 2 (1992), 209–228.
- [95] Gina G. Turrigiano, Kenneth R. Leslie, Niraj S. Desai, Lana C. Rutherford, and Sacha B. Nelson. 1998. Activity-dependent scaling of quantal amplitude in neocortical neurons. *Nature* 391 (1998).
- [96] Gina G. Turrigiano and Sacha B. Nelson. 2004. Homeostatic plasticity in the developing nervous system. *Nature Reviews Neuroscience* 5 (2004).
- [97] M. C. W. van Rossum, G. Q. Bi, and G. G. Turrigiano. 2000. Stable Hebbian Learning from Spike Timing-Dependent Plasticity. *The Journal of Neuroscience* 20, 23 (2000).
- [98] T. P. Vogels, H. Sprekeler, F. Zenke, C. Clopath, and W. Gerstner. 2011. Inhibitory Plasticity Balances Excitation and Inhibition in Sensory Pathways and Memory Networks. *Science* 334 (2011).
- [99] Runchun M. Wang, Chetan S. Thakur, and André van Schaik. 2018. An FPGA-Based Massively Parallel Neuromorphic Cortex Simulator. *Frontiers in Neuroscience* 12 (2018).
- [100] Thomas Willhalm, Roman Dementiev, and Patrick Fay. [n. d.]. Intel® Performance Counter Monitor - A better way to measure CPU utilization. <http://www.intel.com/software/pcm>.
- [101] Esin Yavuz, James Turner, and Thomas Nowotny. 2016. GeNN: a code generation framework for accelerated brain simulations. *Scientific Reports* 6 (2016).
- [102] Friedemann Zenke, Everton J. Agnes, and Wulfram Gerstner. 2015. Diverse synaptic plasticity mechanisms orchestrated to form and retrieve memories in spiking neural networks. *Nature communications* 6 (2015), 6922.
- [103] Friedemann Zenke and Wulfram Gerstner. 2014. Limits to high-speed simulations of spiking neural networks using general-purpose computers. *Frontiers in Neuroinformatics* 8 (2014).
- [104] Friedemann Zenke, Guillaume Hennequin, and Wulfram Gerstner. 2013. Synaptic Plasticity in Neural Networks Needs Homeostasis with a Fast Rate Detector. *PLOS Computational Biology* 9, 11 (2013), e1003330.